Agentic Research & Decision Intelligence Platform

A Multi-Agent AI System for Evidence-Grounded Research and Decision Support

Demonstration Topic: Multi-agent AI systems for trustworthy clinical decision support

System mode:	Deterministic mock mode (`USE_MOCK_LLM=true`)
Primary stack:	Python, LangGraph, FastAPI, Streamlit, SQLite, Pytest
Run ID:	`5b0c4d12-95f5-49bc-a321-d68958dcf9ff`
Final quality score:	93.1/100
Confidence score:	95.5/100

University Final Project Submission

Abstract

This report documents the Agentic Research & Decision Intelligence Platform, a multi-agent system that transforms a complex research query into an evidence-grounded academic report, figures, evaluation artifacts, and presentation. The demonstration topic—trustworthy clinical decision support with multi-agent AI—requires traceable retrieval, claim-level fact checking, critique loops, and explicit human oversight. Unlike a single-prompt chatbot, the platform routes control through typed `WorkflowState`, conditional LangGraph edges, and bounded revision budgets. Ten specialized agents plan, retrieve, analyze, critique, verify, visualize, write, evaluate, present, and persist results. The default execution path is fully reproducible: a local corpus of 21 verified references, deterministic mock mode, Pytest coverage, and one-command demo scripts. Optional extension points support live search and external LLM providers without changing agent contracts.

Contents

1 Introduction **1**

2 Problem Statement **1**

3 Motivation **1**

4 Related Work **1**

5 System Overview **1**

6 Multi-Agent Architecture **2**

7 Agent Roles and Responsibilities **3**

8 LangGraph Workflow Design **4**

9 Agentic Behavior and Conditional Routing **5**

10 State Management and Memory **6**

11 Tool-Use Design **6**

12 Retrieval and Citation Strategy **6**

13 Mathematical Formulation **7**

14 Implementation Details **7**

15 FastAPI Backend Design **7**

16 User Interface and Demonstration **8**

17 Report Generation Pipeline **14**

18 Evaluation Methodology **14**

19 Results **17**

20 Testing and Reproducibility **18**

21 Error Analysis **18**

22 Limitations **18**

23 AI-Assisted Development Disclosure **19**

24 Ethical Considerations **20**

25 Future Work **20**

26 Conclusion **20**

A Framework Comparison **21**

B Conceptual Grounding Figure **21**

C Claim Checks **21**

List of Figures

1 Overall system architecture connecting Streamlit, FastAPI, LangGraph, agents, tools, memory, and final outputs. 2
2 Agent communication through shared structured state rather than hidden chat transcripts. 3
3 Conditional multi-agent workflow with research revision and report evaluation loops. 4
4 LangGraph state transition diagram showing how status values move through the graph. 5
5 Actual Streamlit Overview tab captured in a headless browser after mock demo run. 9
6 Actual Streamlit Workflow tab captured in a headless browser after mock demo run. 10
7 Actual Streamlit Agents tab captured in a headless browser after mock demo run. . 11
8 Actual Streamlit Results tab captured in a headless browser after mock demo run. . 12
9 Actual Streamlit Figures tab captured in a headless browser after mock demo run. . 13
10 Demo evaluation score summary generated from the mock workflow run. 13
11 Report generation pipeline from retrieved sources to LaTeX/PDF and presentation artifacts. 14
12 Evaluation pipeline with rubric scoring, thresholding, and revision routing. 15
13 Evidence source distribution by theme from the deterministic demo corpus. 17
14 Radar chart summarizing selected quality dimensions for the prototype output. . . . 18
15 Conceptual view of grounded synthesis from question to evidence-backed report. . . 21

List of Tables

1 Agent Roles 3
2 Conditional Routing Rules 5
3 Sample of Retrieved Evidence Sources 6
4 API Endpoints 8
5 Evaluation Rubric 16

6 Final Evaluation Scores 17
7 Limitations and Mitigations 19
8 AI tools used during development 20
9 Framework Comparison 21

1 Introduction

Research and decision-support tasks require more than fluent text generation. Practitioners need to inspect planning decisions, retrieved evidence, intermediate tables, critique notes, and citation grounding before trusting a synthesis [12], [18]. This project implements a multi-agent platform that produces those inspectable artifacts for a demanding scenario: multi-agent AI systems for trustworthy clinical decision support. The design prioritizes auditability, reproducibility, and academic presentation quality suitable for university evaluation.

2 Problem Statement

Given a broad research or decision scenario, the system must automatically produce a professional evidence-grounded report and presentation. For the demo query, the platform must explain how multi-agent orchestration can improve clinical decision support while reducing hallucination risk. Concrete requirements include task decomposition, ranked evidence retrieval, structured comparison tables, critique with optional research revision, claim-level fact checking, figure generation, LaTeX/PDF report compilation, rubric-based evaluation with optional report revision, and persistent run memory—all without requiring paid API keys in the default grading path.

3 Motivation

Clinical decision support is a useful motivating domain because errors have high stakes and because prior reviews show that effectiveness depends on workflow integration, timing, and governance rather than model fluency alone [2], [7], [15]. Large language models can appear authoritative while remaining unsupported or outdated [21], [5]. A multi-agent architecture makes control explicit: planning, retrieval, critique, verification, and evaluation are separate responsibilities with inspectable state transitions [19], [8].

4 Related Work

Retrieval-augmented generation grounds generation in external evidence [12]. ReAct interleaves reasoning traces with tool actions [18]. Reflexion and Self-Refine demonstrate verbal reinforcement and iterative self-feedback [10], [1]. AutoGen, CrewAI, Semantic Kernel, and Google ADK represent alternative multi-agent orchestration styles [14], [4], [9], [6]. For clinical AI, reporting frameworks such as CONSORT-AI, SPIRIT-AI, and DECIDE-AI stress transparency and implementation readiness [20], [16], [3]. Automated RAG evaluation methods such as RAGAS inform the rubric design [17].

5 System Overview

The platform spans five layers: Streamlit UI and FastAPI API, graph orchestration, ten specialized agents, deterministic tool adapters, and SQLite plus filesystem outputs. Figure 1 summarizes the architecture. Synchronous endpoints suit the course-demo scale; the explicit state model supports future job queues and asynchronous execution.

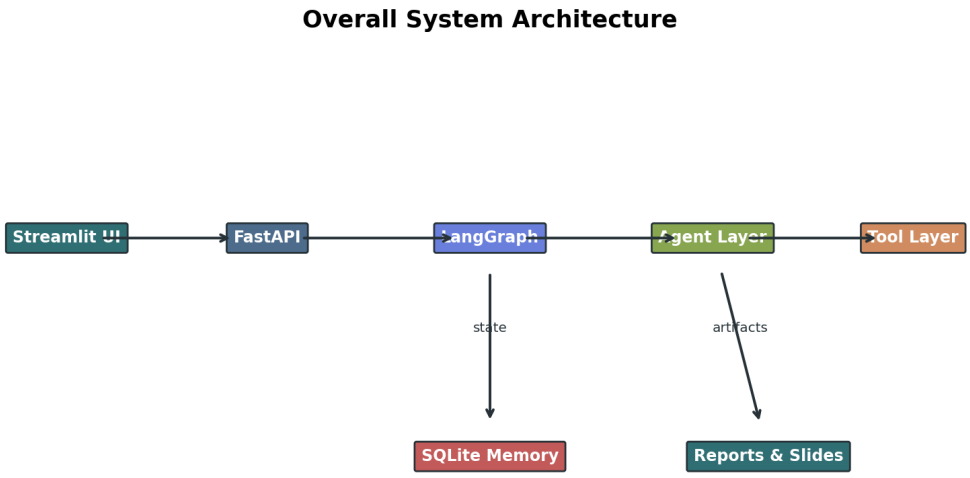

Figure 1: Overall system architecture connecting Streamlit, FastAPI, LangGraph, agents, tools, memory, and final outputs.

6 Multi-Agent Architecture

Ten agents collaborate through `WorkflowState`, a typed container for the query, plan, sources, tables, critique, claim checks, figures, artifact paths, evaluation scores, and agent notes. Figure 2 shows that agents exchange structured fields rather than hidden chat transcripts, which improves testability and grading transparency.

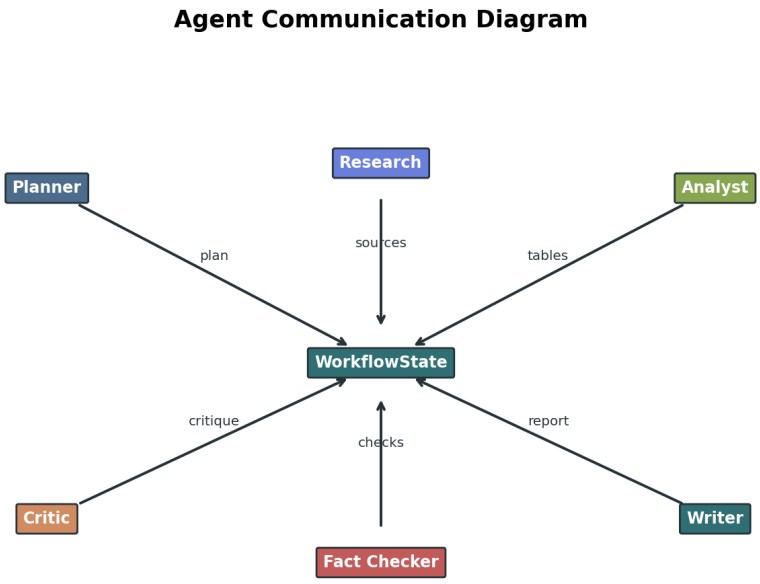

Figure 2: Agent communication through shared structured state rather than hidden chat transcripts.

Table 1: Agent Roles

Agent	Responsibility	Input	Output
Planner	Task decomposition and routing	User query	Execution plan (PlanStep list)
Research	Evidence retrieval	Query and plan	Ranked Source objects
Analyst	Metrics, comparisons, formulas	Retrieved sources	Analysis tables and metrics
Critic	Weakness and risk review	Analysis and sources	needs_revision flag and critique
Fact-Checker	Claim grounding	Approved analysis	ClaimCheck objects with citations
Visualization	Figures and diagrams	Workflow state snapshot	PNG, SVG, and Mermaid assets
Report Writer	Academic synthesis	Full workflow state	Markdown, LaTeX, and PDF reports
Evaluator	Rubric scoring	Report and figures	EvaluationScore JSON/-Markdown
Presentation	Stakeholder communication	Evaluated state	Markdown and PPTX slides
Memory	Persistence	Completed workflow state	SQLite run record

7 Agent Roles and Responsibilities

The **Planner** decomposes the user query into subtasks and assigns agents. The **Research** agent retrieves ranked sources from the mock corpus or optional external search tools. The **Analyst**

computes coverage metrics, comparison tables, and formula blocks. The **Critic** inspects evidence sufficiency and can set `needs_revision`. The **Fact-Checker** maps claims to citation markers from retrieved sources only. The **Visualization** agent renders architecture diagrams and evaluation charts. The **Report Writer** emits Markdown, LaTeX, and PDF deliverables. The **Evaluator** applies the rubric in Table 5. The **Presentation** agent produces slide artifacts. The **Memory** agent persists run metadata to SQLite.

8 LangGraph Workflow Design

LangGraph (or a deterministic fallback runner with identical routing rules) executes the workflow as a state graph. Figure 3 shows agent ordering and revision loops; Figure 4 shows status transitions. Conditional edges are the primary mechanism that makes the system agentic rather than a fixed prompt chain [8].

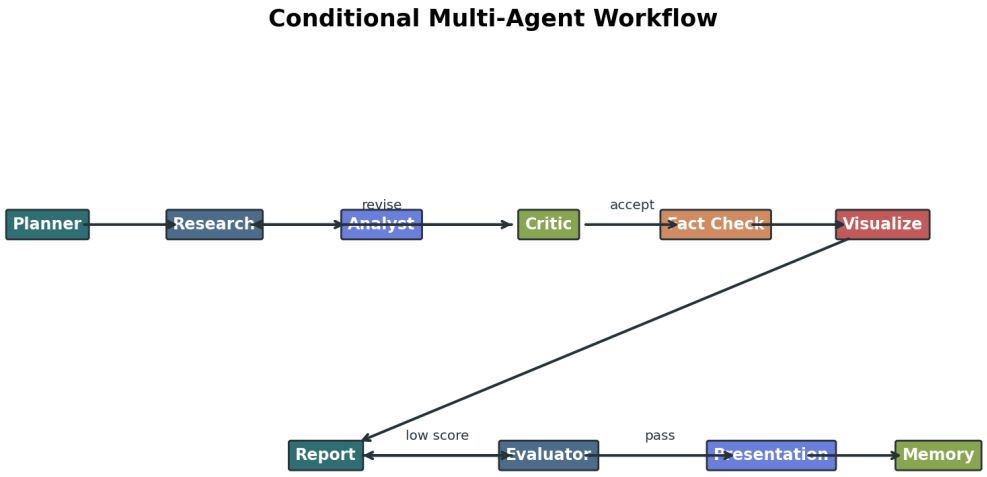

Figure 3: Conditional multi-agent workflow with research revision and report evaluation loops.

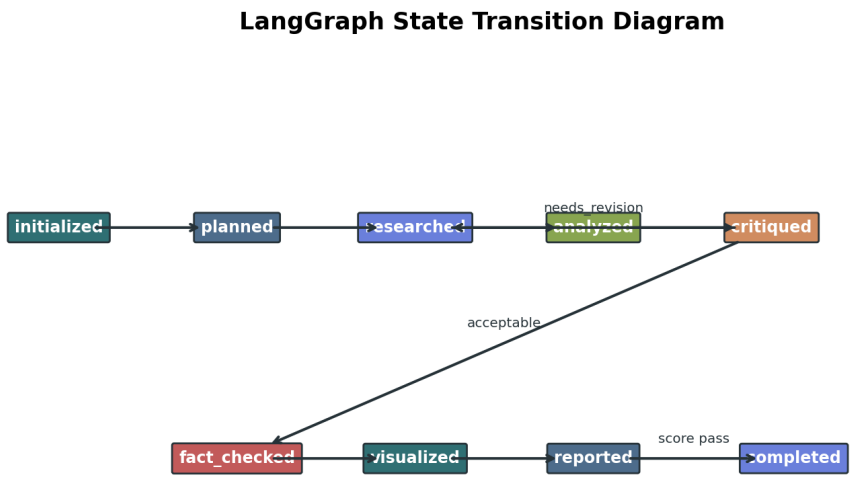

Figure 4: LangGraph state transition diagram showing how status values move through the graph.

Table 2: Conditional Routing Rules

Decision Point	Condition	Next Node
Critic	Evidence is insufficient and revision budget remains	Research Agent
Critic	Evidence is acceptable	Fact-Checking Agent
Evaluator	Quality score $Q ¡ \tau$ (78)	Report Writer Agent
Evaluator	Quality score is acceptable	Finalize Report then Presentation
Presentation	Presentation generated	Memory Agent

9 Agentic Behavior and Conditional Routing

Agentic behavior here means routing decisions depend on structured state, not only on the latest model token. After analysis, `route_after_critic` returns `research` when evidence is weak and revision budget remains; otherwise execution proceeds to fact checking. After report writing, `route_after_evaluator` returns `report` when $Q < \tau$ with $\tau = 78$; otherwise execution proceeds to presentation. Listing 1 shows the core routing logic implemented in `app/graph/nodes.py`.

Listing 1: Core conditional routing logic

```
def route_after_critic(state):
    if state.needs_revision and state.revision_count <= state.max_revisions:
        return "research"
    return "fact_check"
```

```
def route_after_evaluator(state):
    if state.evaluation.total < 78 and state.revision_count < state.max_revisions:
        state.revision_count += 1
        return "report"
    return "presentation"
```

10 State Management and Memory

`WorkflowState` is serializable and validated with Pydantic when available, with a compatibility shim for constrained environments. SQLite stores run IDs, artifact paths, evaluation totals, timestamps, and serialized state for API inspection via `GET /runs/{run_id}`. Memory is used for auditability, not hidden cross-run prompt conditioning, which keeps mock-mode grading deterministic.

11 Tool-Use Design

Tools are small deterministic functions: `search_tools` for mock or external retrieval, `citation_tools` for markers, `visualization_tools` for diagrams and charts, `latex_report_tools` for academic PDF compilation, and `file_tools` for safe path handling. Agents remain orchestration-focused; tools encapsulate repeatable IO and formatting. This mirrors the tool-use pattern described in ReAct [18].

12 Retrieval and Citation Strategy

The offline corpus contains 21 references with DOI or official documentation URLs. Mock retrieval ranks sources by keyword overlap and credibility, yielding deterministic outputs for tests. The fact checker never invents citations; it only links claims to `Source` objects already present in state. Top evidence themes in the demo run were: evaluation (9), clinical (8), agentic (8), ethics (6), framework (5), risk (4), decision_support (3), reporting (3).

Table 3: Sample of Retrieved Evidence Sources

Type	Year	Credibility	Tags
paper	2020	0.96	clinical, decision_support, risk
paper	2005	0.94	clinical, decision_support, evaluation
paper	2005	0.94	clinical, decision_support, evaluation
guideline	2022	0.96	clinical, ethics, reporting
documentation	2026	0.84	agentic, framework, adk
policy	2021	0.98	clinical, ethics, governance
guideline	2020	0.97	clinical, ethics, evaluation
guideline	2020	0.97	clinical, ethics, reporting
paper	2023	0.95	clinical, medical, evaluation
paper	2023	0.91	hallucination, evaluation, risk

13 Mathematical Formulation

The Planner decomposes a high-level user query into subtasks:

$$T = \{t_1, t_2, \ldots, t_n\} \tag{1}$$

Each t_i maps to an agent responsibility and optional dependency edges in the execution plan.

The Analyst aggregates weighted evidence signals into a confidence score:

$$C = \frac{\sum_{i=1}^{n} w_i s_i}{\sum_{i=1}^{n} w_i} \tag{2}$$

where s_i is a quality signal (for example source credibility or coverage) and w_i is its weight. The demo confidence score is 95.5/100, computed from credibility, citation coverage, and tag diversity.

The Evaluator combines rubric dimensions into an overall quality score:

$$Q = \alpha F + \beta R + \gamma S + \delta C_q \tag{3}$$

where F is factuality, R is relevance, S is structure, and C_q is citation quality. The implementation extends this with completeness, clarity, visual quality, and reproducibility dimensions summarized in Table 6.

The graph uses a thresholded revision decision:

$$r = \begin{cases} 1, & Q < \tau \\ 0, & Q \geq \tau \end{cases} \tag{4}$$

When $r = 1$, the workflow routes back to the Report Writer (subject to revision budget), implementing a quality gate before presentation generation.

14 Implementation Details

The repository is organized into `app/agents`, `app/graph`, `app/tools`, `app/models`, `app/storage`, `app/api`, and `app/ui`. LangGraph compiles when installed; otherwise the fallback runner in `app/graph/workflow.py` executes identical conditional logic. External providers are configuration-driven—no API keys are hardcoded. Report compilation prefers `latexmk` and falls back to `pdflatex` plus `bibtex` via `scripts/build_report_pdf.py`.

15 FastAPI Backend Design

FastAPI exposes a minimal synchronous API for demo and grading. Table 4 lists endpoints. `POST /run` accepts a JSON query payload, executes the workflow, and returns run metadata including artifact paths and evaluation score. File endpoints stream Markdown reports, presentations, and figure listings.

Table 4: API Endpoints

Endpoint	Method	Purpose
/health	GET	Service readiness check
/run	POST	Execute the full agentic workflow
/runs/{run_id}	GET	Retrieve stored run metadata and serialized state
/runs/{run_id}/report	GET	Download the generated Markdown report
/runs/{run_id}/presentation	GET	Download the presentation artifact
/runs/{run_id}/figures	GET	List paths to generated figure files

16 User Interface and Demonstration

A Streamlit interface (`app/ui/streamlit_app.py`) provides a professor-friendly demo surface alongside the FastAPI API. The UI allows evaluators to enter a topic, launch the multi-agent workflow, inspect agent outputs, review evaluation scores, preview generated figures, and download report/presentation artifacts without writing code.

Why a UI was added: The project is easier to grade and demonstrate when workflow behavior, intermediate outputs, and final artifacts are visible in one place. The interface exposes typed agent notes, rubric scores, figure previews, and download buttons rather than hiding collaboration inside a single chat box.

Tabs: Overview (run metrics and artifact status), Workflow (pipeline timeline and conditional routing), Agents (agent cards), Results (rubric breakdown and progress bars), Figures (gallery preview), Downloads (PDF/Markdown/PPTX/JSON), and About (agentic design and limitations).

Mock mode: The sidebar defaults to deterministic mock execution using the local evidence corpus, supporting reproducible grading without paid API keys. A visible warning states that the clinical scenario is academic only.

Screenshot note: UI images in this report are browser captures. Metadata: Captured with Playwright/Chromium on port 8511 after seeded mock demo..

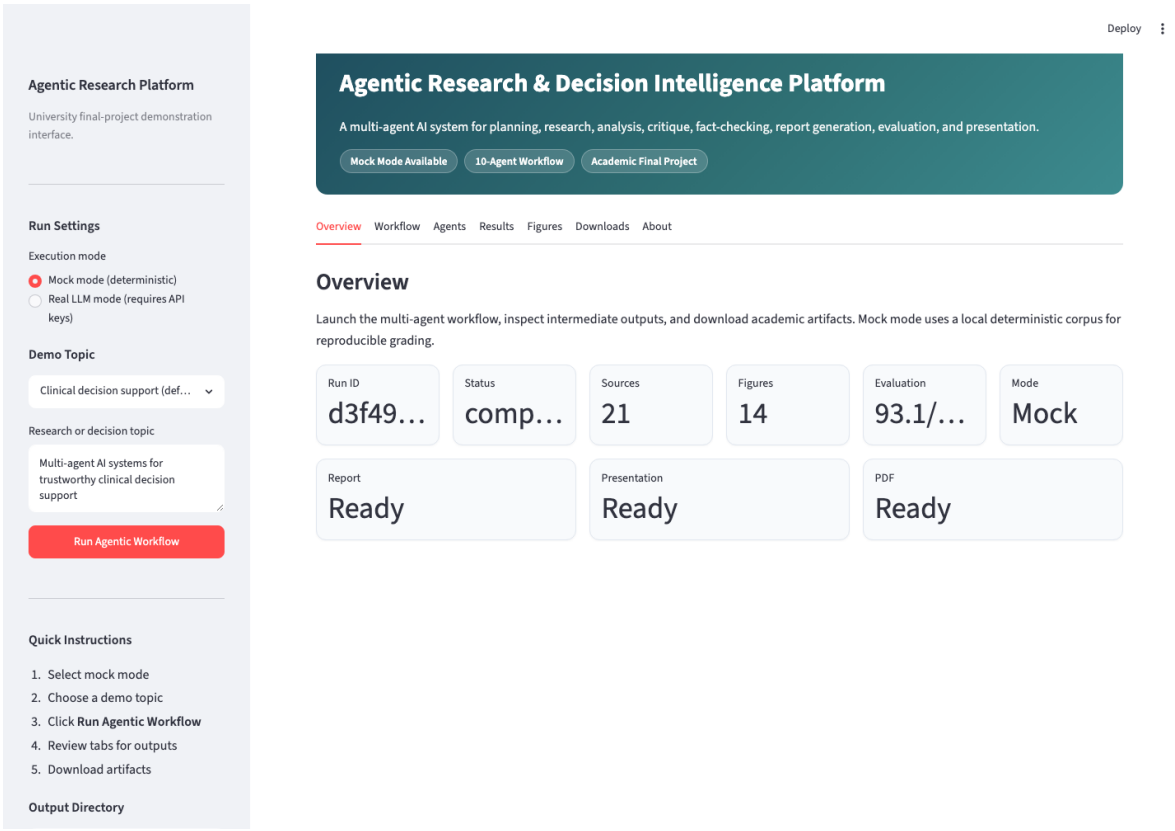

Figure 5: Actual Streamlit Overview tab captured in a headless browser after mock demo run.

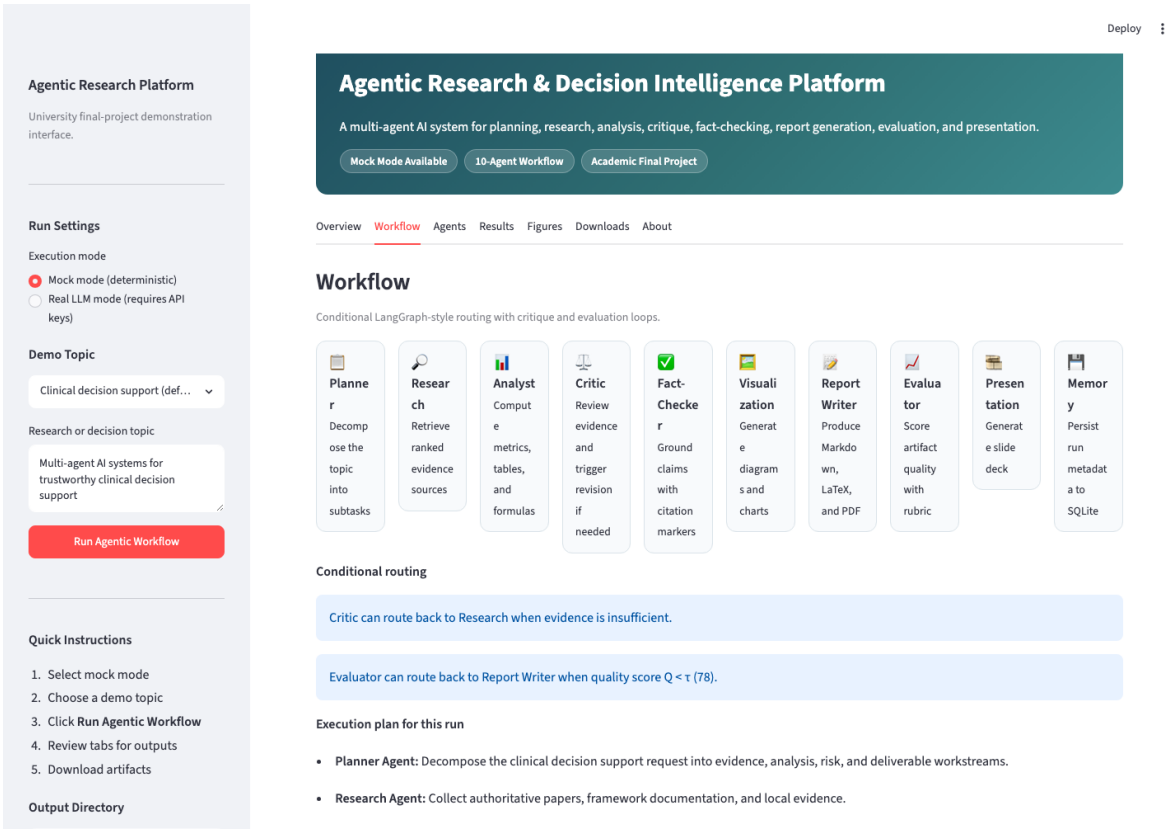

Figure 6: Actual Streamlit Workflow tab captured in a headless browser after mock demo run.

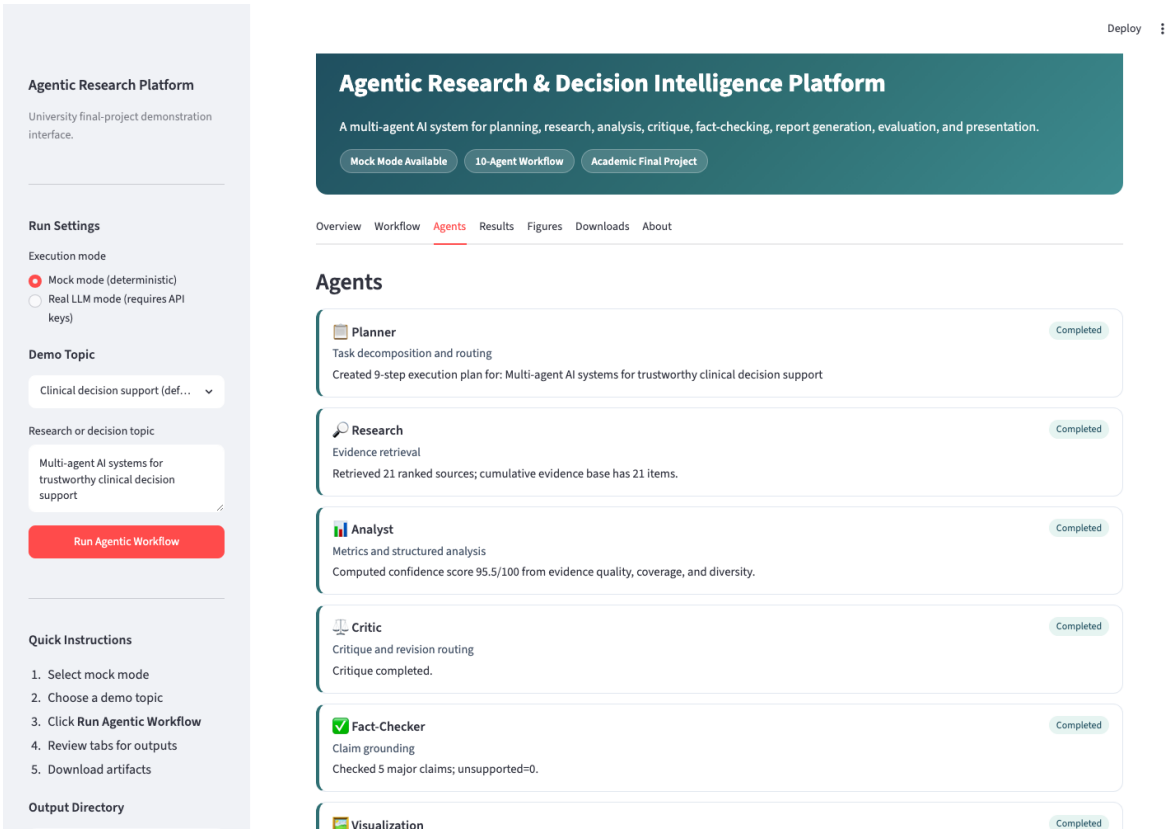

Figure 7: Actual Streamlit Agents tab captured in a headless browser after mock demo run.

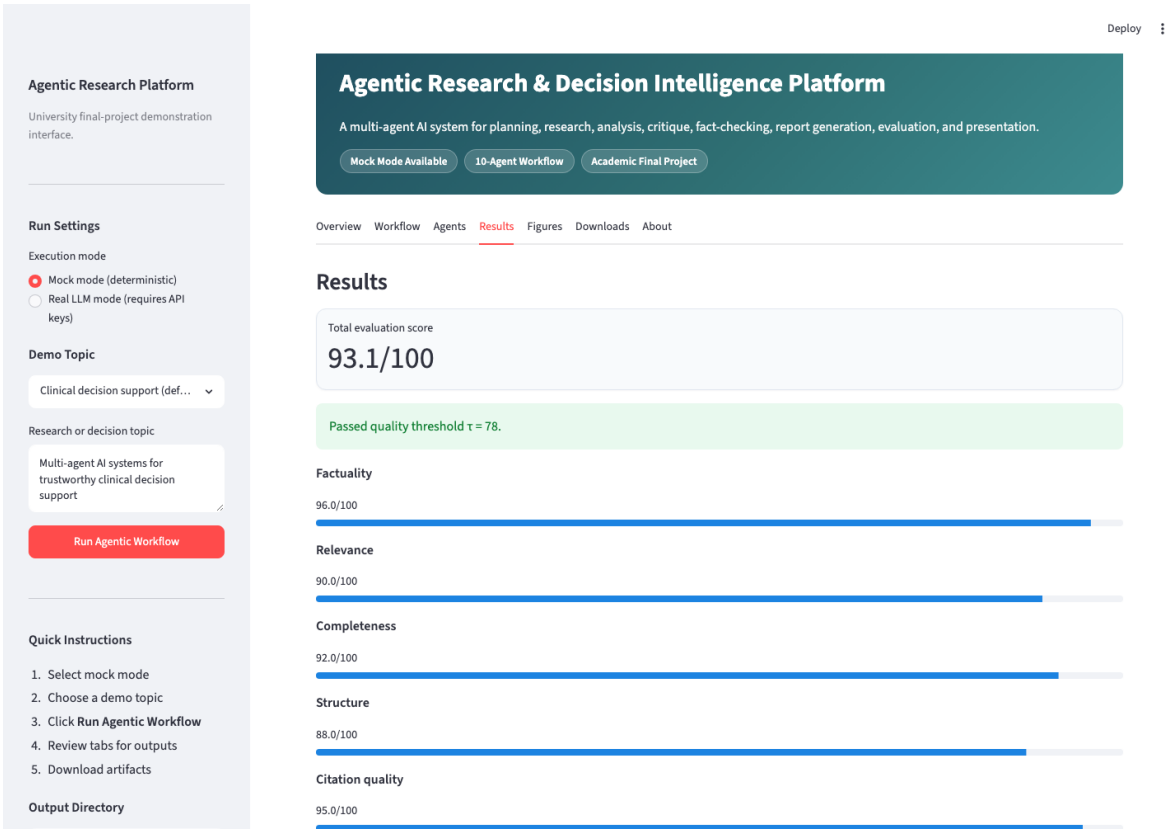

Figure 8: Actual Streamlit Results tab captured in a headless browser after mock demo run.

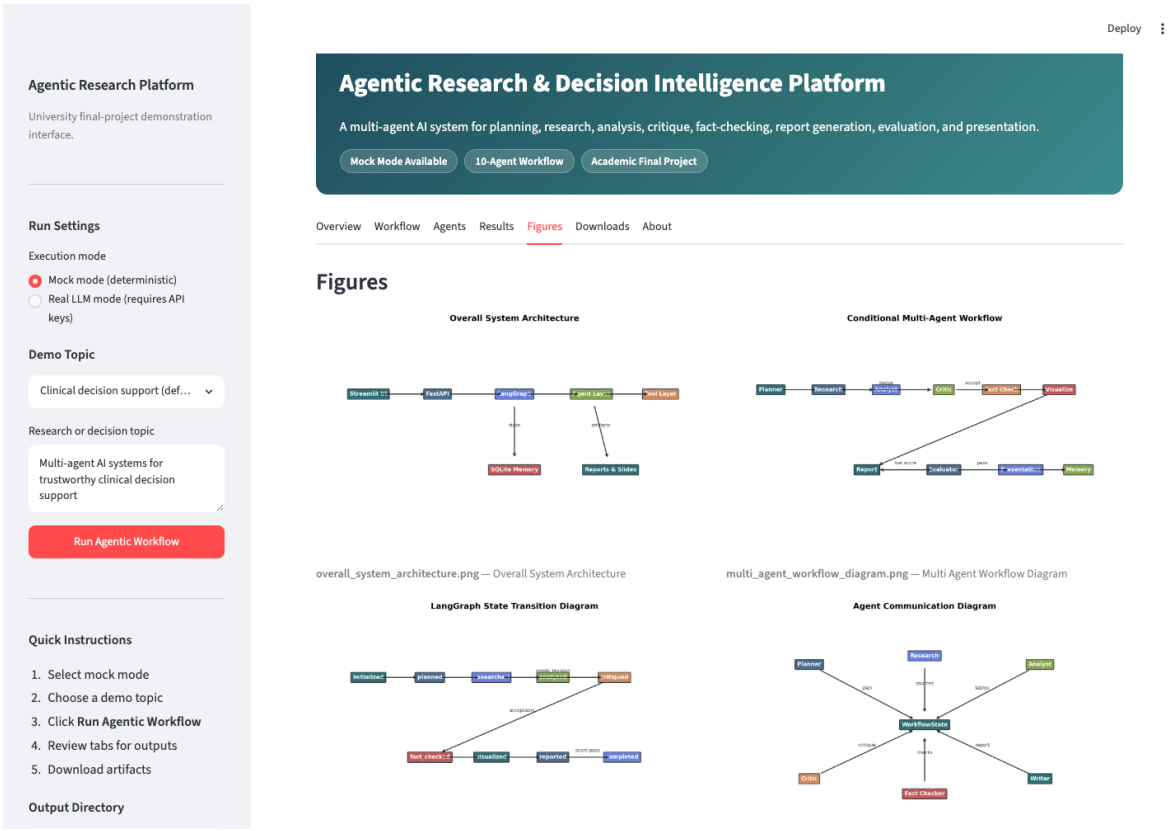

Figure 9: Actual Streamlit Figures tab captured in a headless browser after mock demo run.

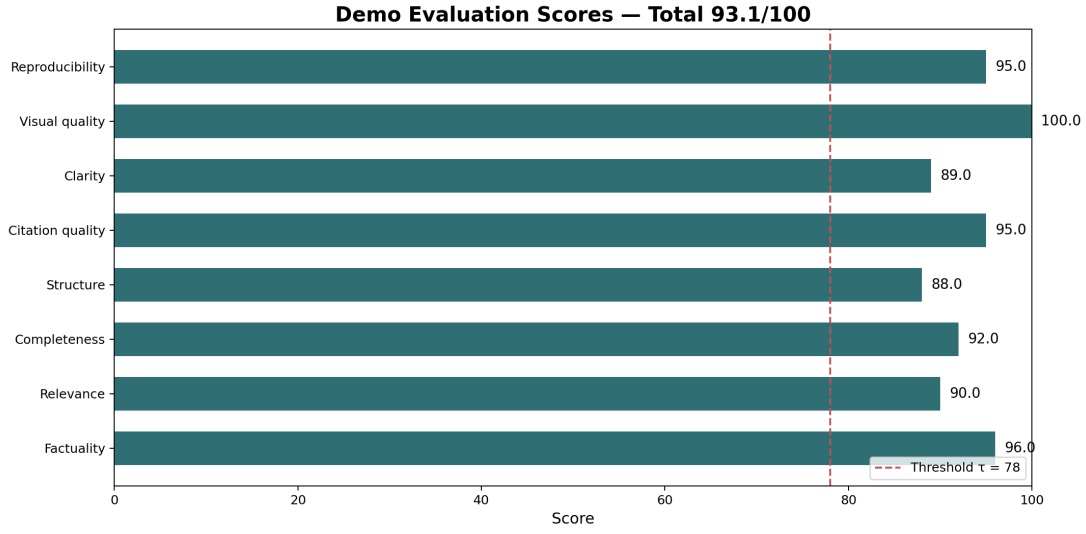

Figure 10: Demo evaluation score summary generated from the mock workflow run.

Launch: `streamlit run app/ui/streamlit_app.py`. Capture assets: `python scripts/capture_ui_screens`
See `docs/SCREENSHOT_GUIDE.md`.

17 Report Generation Pipeline

The Report Writer assembles Markdown and LaTeX from tables, formulas, figures, critiques, and claim checks. Figure 11 shows the pipeline. Canonical deliverables are written to `report/final_report.tex`, `report/final_report.pdf`, and `report/final_report.md`; run-specific copies are stored under `outputs/reports/`.

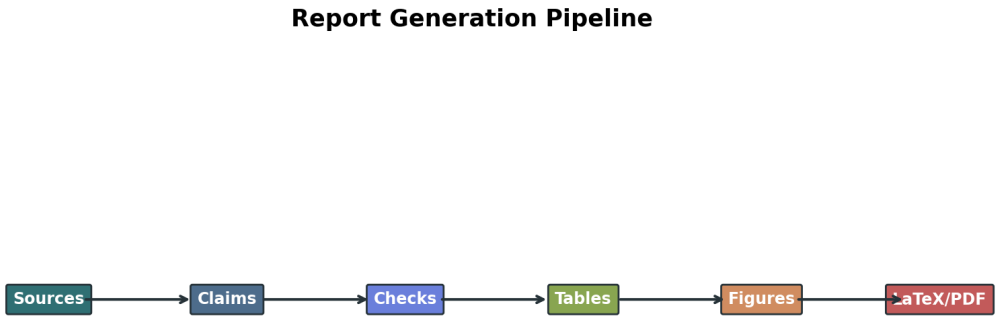

Figure 11: Report generation pipeline from retrieved sources to LaTeX/PDF and presentation artifacts.

18 Evaluation Methodology

The Evaluator scores eight rubric dimensions from 0 to 100 using transparent heuristics tied to claim support, artifact completeness, figure count, and reproducibility features. Table 5 defines criteria and weights. Figure 12 shows threshold gating. The score measures project artifact quality, not clinical validity [17].

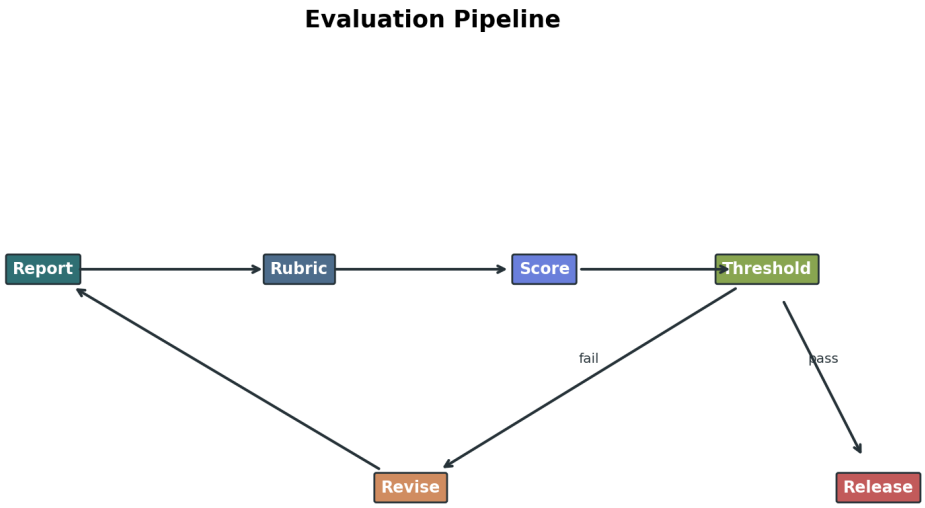

Figure 12: Evaluation pipeline with rubric scoring, thresholding, and revision routing.

Table 5: Evaluation Rubric

Criterion	Description	Weight
Factuality	Share of claims supported by retrieved sources	15%
Relevance	Alignment between query, plan, and final report	12%
Completeness	Presence of required sections, tables, figures, and formulas	12%
Structure	Logical organization and academic readability	10%
Citation quality	Traceable references and citation markers	15%
Clarity	Plain-language explanations of technical decisions	10%
Visual quality	Architecture diagrams, charts, and labeled figures	13%
Reproducibility	Mock mode, tests, scripts, Docker, and saved artifacts	13%

Table 6: Final Evaluation Scores

Dimension	Score
Factuality	96.0
Relevance	90.0
Completeness	92.0
Structure	88.0
Citation quality	95.0
Clarity	89.0
Visual quality	100.0
Reproducibility	95.0
Total	93.1

19 Results

The final mock run retrieved 21 sources, generated 14 figures, compiled a LaTeX PDF, produced a presentation, and saved evaluation JSON/Markdown under `outputs/evaluation/`. The final quality score was 93.1/100. Figure 13 shows corpus theme distribution; Figure 14 summarizes rubric dimensions.

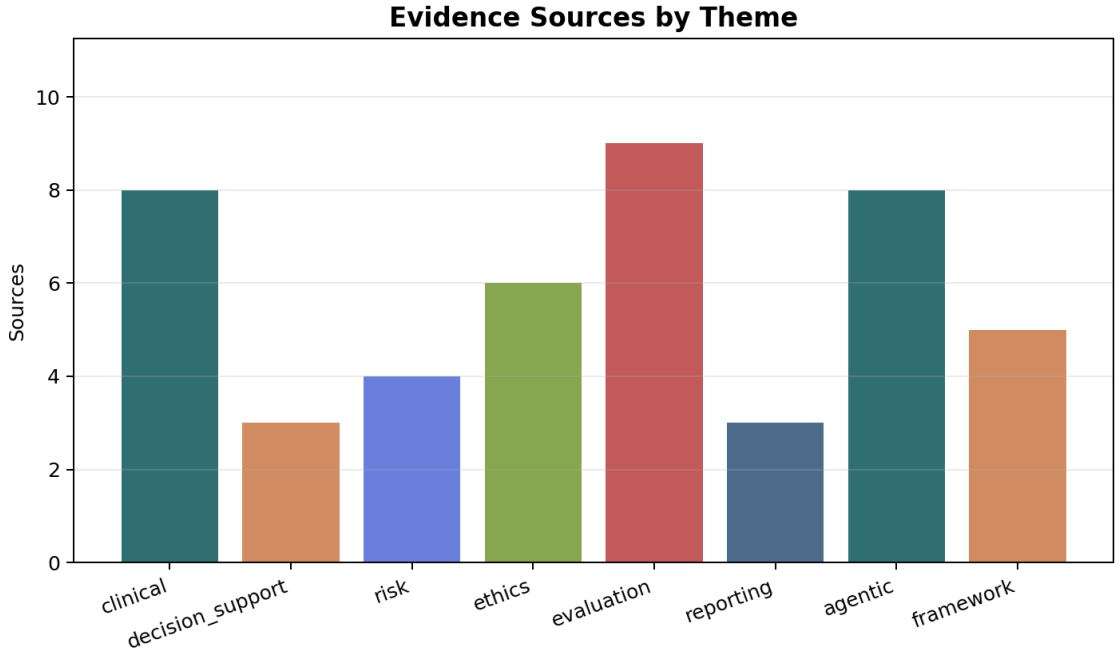

Figure 13: Evidence source distribution by theme from the deterministic demo corpus.

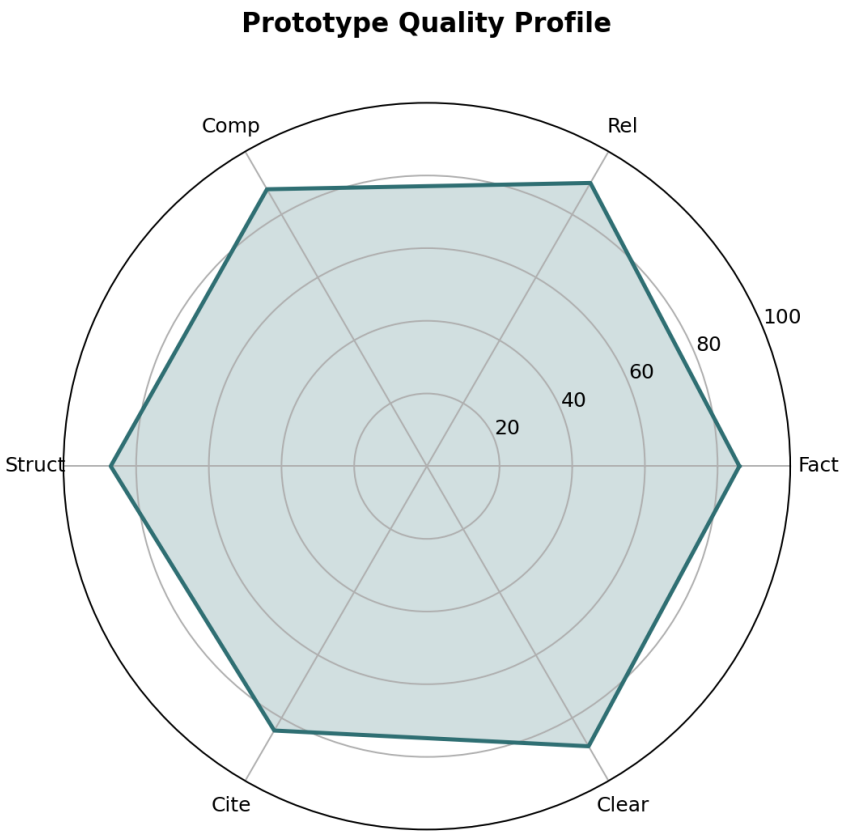

Figure 14: Radar chart summarizing selected quality dimensions for the prototype output.

20 Testing and Reproducibility

Pytest covers unit tests for each core agent, integration tests for workflow and API behavior, and an end-to-end demo test. Primary verification commands are `python -m compileall app scripts`, `python scripts/run_demo.py`, `python scripts/build_report_pdf.py`, and `pytest -v`. Docker Compose files support containerized execution. Mock mode (`USE_MOCK_LLM=true`) is the supported grading default [13].

21 Error Analysis

Expected failure modes include stale evidence, incomplete retrieval, weak claim-to-source alignment, shallow critique, and overconfident clinical wording. Mitigations include retrieval grounding, explicit critique, metadata-level claim checks, limitations sections, evaluator thresholds, and mandatory human review for high-stakes use. Production deployments should add passage-level entailment and source freshness scoring [21].

22 Limitations

Table 7 summarizes prototype limitations and mitigations. The system is not clinically validated and must not be used for patient-specific diagnosis or treatment [11].

Table 7: Limitations and Mitigations

Limitation	Impact	Mitigation
Static mock corpus	Evidence may be stale or incomplete	Deterministic grading; optional live search adapters
Real LLM dependency in production	Non-deterministic outputs without keys	USE_MOCK_LLM=true default; provider adapters behind tools
Citation verification limits	Tag-level checks miss entailment errors	Fact-checker gates claims; human review required
Clinical prototype scope	Not validated for patient-specific decisions	Explicit limitations, ethics section, no diagnosis claims
Human review requirement	Automation bias if outputs are trusted blindly	Critic mandates limitations; evaluator threshold gate

23 AI-Assisted Development Disclosure

This submission was developed with external AI engineering assistants used transparently as development aids. ChatGPT supported project planning, architecture discussions, prompt engineering, documentation drafting, and report structure. A Codex-style coding assistant supported source code, tests, scripts, and documentation generation. Cursor AI served as the AI-assisted IDE for refactoring, LaTeX/PDF polishing, and consistency validation. NotebookLM assisted presentation drafting and speaker notes. Equivalent tools may substitute, but the submitted repository must remain runnable without them.

AI tools were used as development assistants. The student directed project requirements, reviewed generated outputs, tested the system, selected the architecture, validated deliverables, and prepared the final submission. The student remains responsible for correctness and academic integrity.

Table 8 summarizes external development tools. These are distinct from the ten **internal runtime agents** (Planner, Research, Analyst, Critic, Fact-Checker, Visualization, Report Writer, Evaluator, Presentation, Memory) orchestrated by LangGraph when the platform executes. Development AI helped build the project; internal agents are part of the implemented system documented in `docs/AI_GUIDE.md` and `docs/AI_USAGE_GUIDE.md`.

Table 8: AI tools used during development

Tool	Purpose	Human Oversight
ChatGPT	Planning, architecture, documentation, prompt engineering	Student reviewed and refined outputs
Codex-style assistant	Code generation, tests, scripts, docs	Student tested and validated implementation
Cursor AI	Refactoring, report polishing, LaTeX improvement	Student supervised edits and verified behavior
NotebookLM	Presentation drafting and speaker notes	Student reviewed and adapted slides

24 Ethical Considerations

Clinical AI introduces automation bias, privacy risk, inequitable recommendations, and misplaced trust in fluent prose [11], [5]. This prototype is a research engineering artifact. Deployment would require PHI controls, institutional governance, security review, monitoring, audit logs, and explicit clinician responsibility.

25 Future Work

Planned extensions include live search adapters, vector retrieval over institution-approved corpora, stronger natural-language entailment for fact checking, human-in-the-loop approval checkpoints, asynchronous job queues, authentication, and formal evaluation against clinical reporting standards such as DECIDE-AI [3].

26 Conclusion

The Agentic Research & Decision Intelligence Platform demonstrates how graph-based multi-agent orchestration can produce transparent, evidence-grounded academic deliverables. Its contribution is the combination of typed state, conditional critique and evaluation loops, deterministic mock mode, professional LaTeX reporting, and reproducible verification scripts suitable for university final-project evaluation.

A Framework Comparison

Table 9: Framework Comparison

Framework	Strengths	Limitations	Suitability for This Project
LangGraph	Typed graph orchestration with conditional edges	Requires explicit state design	Best fit: mirrors critique and evaluation loops
Google ADK	Code-first agent toolkit with Google ecosystem deployment	Younger ecosystem and vendor coupling	Useful for Gemini/Vertex deployments, not required here
AutoGen	Conversational multi-agent patterns	Can become chat-loop centric	Good for research prototypes; less explicit graph control
CrewAI	Role/task abstraction is easy to teach	Less explicit low-level routing	Suitable for sequential crews, not revision gates
Semantic Kernel	Enterprise-friendly plugins and planners	Broad SDK surface area	Strong for Microsoft stacks; heavier than needed

B Conceptual Grounding Figure

Grounded decision intelligence: every recommendation must trace back to evidence, critique, and citation checks.

Figure 15: Conceptual view of grounded synthesis from question to evidence-backed report.

C Claim Checks

- **Supported**: Retrieval grounding can reduce unsupported generation by forcing reports to cite external evidence.
- **Supported**: Clinical decision support requires human oversight and careful workflow integration.
- **Supported**: Conditional multi-agent graphs make critique and revision loops explicit.
- **Supported**: Evaluation should include factuality, relevance, structure, citation quality, and reproducibility.

- **Supported**: The prototype is not a medical device and should not produce patient-specific diagnosis.

References

[1] Prakhar Gupta-et al. Aman Madaan, Niket Tandon. Self-refine: Iterative refinement with self-feedback. 2023. Demonstrates iterative self-feedback and refinement for improving generated outputs without extra training.

[2] Heather McDonald et al. Amit X. Garg, Neill K. J. Adhikari. Effects of computerized clinical decision support systems on practitioner performance and patient outcomes. 2005. Systematic review of computerized clinical decision support effects on practitioner performance and patient outcomes.

[3] et al. Brennan Vasey, Xiaoxuan Liu. Decide-ai: New reporting guidelines to bridge the development-to-implementation gap in clinical artificial intelligence. 2022. Offers early-stage reporting guidance for clinical AI systems before full clinical trials.

[4] CrewAI. Crewai documentation, 2026. Framework documentation for role-based agents, crews, tools, memory, and workflows.

[5] Angelina McMillan-Major Shmargaret Shmitchell Emily M. Bender, Timnit Gebru. On the dangers of stochastic parrots: Can language models be too big? 2021. Discusses risks of large language models including bias, environmental cost, and misleading fluent text.

[6] Google. Agent development kit multi-agent systems documentation, 2026. Google ADK documentation for composing multiple agents into multi-agent systems.

[7] E. Andrew Balas David F. Lobach Kensaku Kawamoto, Caitlin A. Houlihan. Improving clinical practice using clinical decision support systems: A systematic review of trials. 2005. Identifies features associated with effective clinical decision support interventions.

[8] LangChain. Langgraph graph api documentation, 2026. Official documentation for LangGraph state graphs, nodes, edges, conditional routing, and workflow control.

[9] Microsoft. Semantic kernel agent framework documentation, 2026. Microsoft documentation for agent abstractions and collaboration patterns in Semantic Kernel.

[10] Ashwin Gopinath et al. Noah Shinn, Federico Cassano. Reflexion: Language agents with verbal reinforcement learning. 2023. Presents a reflection loop for agents that critique prior attempts and improve subsequent actions.

[11] World Health Organization. Ethics and governance of artificial intelligence for health, 2021. WHO guidance on ethical, legal, governance, and human-rights issues for AI in health.

[12] Aleksandra Piktus et al. Patrick Lewis, Ethan Perez. Retrieval-augmented generation for knowledge-intensive nlp tasks. 2020. Introduces retrieval-augmented generation, combining parametric generation with retrieved non-parametric memory for knowledge-intensive tasks.

[13] pytest dev. pytest: Getting started documentation, 2026. Official pytest documentation for writing and running reproducible Python test suites.

[14] Jieyu Zhang et al. Qingyun Wu, Gagan Bansal. Autogen: Enabling next-gen llm applications via multi-agent conversation. 2023. Introduces AutoGen, a framework for building LLM applications through multi-agent conversation.

[15] Daniel C. Baumgart et al. Reed T. Sutton, David Pincock. An overview of clinical decision support systems: Benefits, risks, and strategies for success. 2020. Reviews clinical decision support benefits, implementation risks, human factors, alert fatigue, and governance requirements.

[16] An-Wen Chan et al. Samantha Cruz Rivera, Xiaoxuan Liu. Spirit-ai extension: Protocol guidelines for clinical trials of artificial intelligence interventions. 2020. Defines protocol reporting items for clinical trials involving AI interventions.

[17] Luis Espinosa-Anke Steven Schockaert Shahul Es, Jithin James. Ragas: Automated evaluation of retrieval augmented generation. 2023. Defines evaluation dimensions for retrieval-augmented generation including faithfulness, answer relevance, and context relevance.

[18] Dian Yu-et al. Shunyu Yao, Jeffrey Zhao. React: Synergizing reasoning and acting in language models. 2022. Shows how language models can interleave reasoning traces with tool actions for more grounded task solving.

[19] Lilian Weng. Llm powered autonomous agents, 2023. Technical overview of planning, memory, and tool use in autonomous LLM agents.

[20] David Moher et al. Xiaoxuan Liu, Samantha Cruz Rivera. Consort-ai extension: Reporting guidelines for clinical trials of artificial intelligence interventions. 2020. Provides reporting standards for clinical trials evaluating AI interventions.

[21] Rita Frieske et al. Ziwei Ji, Nayeon Lee. Survey of hallucination in natural language generation. 2023. Surveys hallucination definitions, causes, and mitigation methods across natural language generation systems.